

# DevOps Practical Round — Laravel CI/CD on AWS (EC2 + Docker)

## Objective

Design and implement a production-grade CI/CD pipeline for deploying a Laravel application from GitHub to AWS using Dockerized workloads on EC2 instances with autoscaling, zero-downtime deployments, centralized logging, and monitoring.

Candidates are expected to use their own GitHub account/repository and AWS account/infrastructure.

The Laravel application source code must always be pulled from GitHub as part of the deployment workflow. The latest code from the main branch should be automatically deployed through the CI/CD pipeline.

## Infrastructure Requirements

The solution must be deployed on AWS using:

## Mandatory Technical Requirements

CI/CD Pipeline:

- Automated build process
- Retrieval of latest Laravel source code from GitHub
- Docker image creation
- Automated deployment to AWS
- Deployment trigger on push to main branch
- Zero downtime deployment strategy
- Health validation during deployment

Preferred:

- Rollback mechanism
- Deployment versioning

## Deployment & Automation Flexibility

Candidates are free to use any deployment and automation tooling they prefer, including but not limited to:

- Ansible
- Shell scripting
- Terraform
- Packer
- CloudFormation
- Pulumi
- Makefiles
- Custom deployment frameworks

The evaluation will focus on architecture quality, automation maturity, operational reliability, and maintainability rather than the specific tooling selected.

## **Containerization**

Required:

- Production-ready Dockerfile
- nginx + php-fpm setup
- Queue worker support
- Scheduler/cron support

Preferred:

- Multi-stage Docker builds
- Optimized image size
- Separate worker containers/processes

## **AWS Infrastructure Expectations**

Candidates must configure:

- EC2 instances
- Auto Scaling Group
- Application Load Balancer
- Security Groups
- Health checks
- HTTPS configuration (preferred)

The deployment should demonstrate:

- Scalability
- High availability
- Fault tolerance
- Minimal manual operational dependency

## **Monitoring & Logging**

Accepted solutions include:

- CloudWatch
- Grafana
- Prometheus
- Loki
- ELK stack

The solution should provide:

- Application logs
- Infrastructure logs
- Resource monitoring
- Health visibility

## Application Requirements

The deployment must support:

- Laravel web application
- Queue workers
- Scheduler/cron execution
- Environment configuration handling

Optional:

- Redis integration
- Database migrations during deployment

## Candidate Deliverables

Candidates must submit:

- GitHub repository
- GitHub Actions workflows
- Docker configuration
- Deployment scripts
- Architecture diagram
- Deployment documentation
- Public application URL
- Monitoring/logging access details
- README containing:
  - Setup steps
  - Deployment flow
  - Rollback process
  - Scaling approach

## Bonus Points

Additional consideration will be given for:

- Blue/Green deployment
- Automated rollback
- Secret management best practices
- Cost optimization
- Infrastructure as Code
- Deployment observability enhancements
- Security hardening

## Optional Troubleshooting Round

Candidates may additionally be asked to diagnose and resolve issues such as:

- Failed deployment pipeline
- Container startup failures
- nginx misconfiguration
- Queue worker failures
- Health check failures

- Auto Scaling issues
- Permission-related deployment failures

## Expected Outcome

A successful submission should demonstrate:

- Fully automated deployment pipeline
- Production-grade deployment practices
- Reliable autoscaling architecture
- Operational observability
- Stable zero-downtime deployment workflow
- Maintainable and scalable system design

## Infrastructure Requirements

- EC2 instances
- Auto Scaling Group
- Application Load Balancer (ALB)

Candidates are expected to design a scalable and production-ready deployment architecture without using managed container orchestration platforms such as ECS or EKS.

## Evaluation Criteria

| Area                            | Weight |
|---------------------------------|--------|
| CI/CD Automation                | 25%    |
| AWS Architecture Design         | 20%    |
| Zero Downtime Deployment        | 15%    |
| Docker Best Practices           | 15%    |
| Monitoring & Logging            | 10%    |
| Reliability & Scalability       | 10%    |
| Documentation & Maintainability | 5%     |